Puertas universales y lógica reversible

Genesis Silesky

21 de octubre de 2026

1. Puertas universales y lógica clásica reversible

1.1. Objetivo

Ver que toda puerta de un qubit se arma con rotaciones alrededor de z y de y, cómo se construye su versión controlada y cómo la puerta Toffoli permite hacer cálculos de lógica clásica (AND, OR, XOR…) con un computador cuántico.

1.2. Conceptos previos

Concepto previo: rotaciones y puertas controladas

Ver capítulo 12 (rotaciones R_y y R_z) y la sección **??** (CNOT). Una puerta *controlada* aplica una operación al qubit objetivo solo si el qubit de control vale $|1\rangle$.

1.3. Fundamento teórico

Un algoritmo cuántico es un circuito: puertas conectadas en un orden determinado. Este capítulo muestra que con puertas de uno, dos y tres qubits se puede armar cualquier operación.

Puerta universal de un qubit. Toda puerta de un qubit es una matriz unitaria 2×2 y se puede escribir como una rotación alrededor de z, otra alrededor de y y otra alrededor de z, más una fase global:

$$U = e^{i\alpha} R_z(\beta)\, R_y(\gamma)\, R_z(\delta), \tag{1}$$

con $R_z(\theta) = e^{-i\theta Z/2}$ y $R_y(\theta) = e^{-i\theta Y/2}$. Los ángulos β, γ, δ son los de las rotaciones sobre la esfera de Bloch.

Forma $U = e^{i\alpha} AXBXC$. Se puede escribir U con tres matrices unitarias A, B, C que cumplen $ABC = I$:

$$A = R_z(\beta) R_y(\tfrac{\gamma}{2}), \quad B = R_y(-\tfrac{\gamma}{2}) R_z(-\tfrac{\delta+\beta}{2}), \quad C = R_z(\tfrac{\delta-\beta}{2}). \tag{2}$$

Como $X R_y(\theta) X = R_y(-\theta)$ y $X R_z(\theta) X = R_z(-\theta)$, se obtiene $e^{i\alpha} AXBXC = U$. Esto sirve para la puerta controlada: si el control vale $|0\rangle$ las X no actúan y se aplica $ABC = I$ (no pasa nada); si vale $|1\rangle$ se aplica $AXBXC$, que es U (salvo la fase $e^{i\alpha}$, que se agrega con una puerta de fase en el control). Así, una puerta controlada se arma con puertas de uno y dos qubits. Con n controles se obtiene la puerta $C^n(U)$, que aplica U solo si todos valen $|1\rangle$.

Toffoli. La puerta Toffoli es la $C^2(X)$: tiene dos qubits de control y uno objetivo, y lo invierte solo cuando ambos controles valen $|1\rangle$.

Lógica clásica con puertas cuánticas. Toda puerta lógica clásica se puede convertir en una puerta cuántica reversible hecha con X (NOT), CNOT y Toffoli. Con las entradas A, B y la salida C:

- **AND:** Toffoli con A, B de controles y C (en $|0\rangle$) como objetivo: C pasa a $|1\rangle$ solo si $A = B = 1$.
- **OR:** el objetivo C empieza en $|1\rangle$; se aplica X a A y B, luego la Toffoli, y de nuevo X a A y B. El objetivo pasa a $|0\rangle$ solo si $A = B = 0$.
- **NAND, NOR, XOR...:** se arman con las mismas piezas (en este ejemplo, NAND y NOR añaden una X a la salida; XOR son dos CNOT hacia C).

1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 14 - Puertas universales y logica clasica reversible (Toffoli)

# %% Celda 1: importaciones
import itertools
import numpy as np
from scipy.linalg import expm
from qiskit import QuantumCircuit
# Operator convierte un circuito en su matriz; OneQubitEulerDecomposer encuentra los
# angulos de una puerta de un qubit.
from qiskit.quantum_info import Operator, Statevector, random_unitary
from qiskit.synthesis import OneQubitEulerDecomposer

sx = np.array([[0, 1], [1, 0]], dtype=complex)
sy = np.array([[0, -1j], [1j, 0]], dtype=complex)
sz = np.array([[1, 0], [0, -1]], dtype=complex)


def Rz(t):
    return expm(-1j * t / 2 * sz)


def Ry(t):
    return expm(-1j * t / 2 * sy)


# %% Celda 2: cualquier puerta de un qubit es  U = e^{i a} Rz(b) Ry(g) Rz(d)
# Toda puerta de un qubit es una matriz unitaria 2x2. Siempre se puede escribir como
# una rotacion alrededor de z, luego una alrededor de y y otra alrededor de z, mas una
# fase global e^{i a}. Probamos con una unitaria al azar.
U = random_unitary(2, seed=5).data
# Qiskit calcula los angulos (theta = g, phi, lam) de Rz(phi) Ry(theta) Rz(lam).
theta, phi, lam, fase = OneQubitEulerDecomposer("ZYZ").angles_and_phase(U)
reconstruida = np.exp(1j * fase) * Rz(phi) @ Ry(theta) @ Rz(lam)
print("U se reconstruye con Rz Ry Rz:", np.allclose(U, reconstruida))
print(f"angulos: beta = {phi:.3f}, gamma = {theta:.3f}, delta = {lam:.3f}")

# %% Celda 3: U = e^{i a} A X B X C  con  A B C = I
# Esta descomposicion permite construir la version "controlada" de U (aplicar U solo
# si otro qubit vale 1) con puertas de uno y dos qubits. Se toma:
#   A = Rz(b) Ry(g/2),   B = Ry(-g/2) Rz(-(d+b)/2),   C = Rz((d-b)/2).
beta, gamma, delta = phi, theta, lam
A = Rz(beta) @ Ry(gamma / 2)
B = Ry(-gamma / 2) @ Rz(-(delta + beta) / 2)
C = Rz((delta - beta) / 2)
print("A B C = I      :", np.allclose(A @ B @ C, np.eye(2)))
```

```
print("e^{ia} A X B X C = U:", np.allclose(np.exp(1j * fase) * A @ sx @ B @ sx @ C, U))
# Si el qubit de control vale 0 se aplica A B C = I (no pasa nada); si vale 1, las X
# se activan y se aplica A X B X C = U.

# %% Celda 4: la puerta Toffoli (CCX)
# Tiene tres qubits: dos de control y uno objetivo. El objetivo se invierte solo si
# los dos controles valen 1. En Qiskit el orden de la cadena es 'q2 q1 q0': aqui
# q0 y q1 son los controles y q2 el objetivo.
toffoli = QuantumCircuit(3)
toffoli.ccx(0, 1, 2)
print("Tabla de verdad (entrada q2q1q0 -> salida q2q1q0):")
for bits in itertools.product("01", repeat=3):
    entrada = "".join(bits)
    salida = Statevector.from_label(entrada).evolve(toffoli).probabilities_dict()
    print(f"  {entrada} -> {max(salida, key=salida.get)}")
# Solo cambian 011 y 111: el objetivo (q2) se invierte cuando q0 = q1 = 1.

# %% Celda 5: computacion clasica con puertas cuanticas
# Toda puerta logica clasica se puede reescribir con puertas reversibles: X (NOT),
# CNOT y Toffoli. Las entradas son A y B (q0 y q1); la salida queda en C (q2).
def circuito_logico(nombre):
    qc = QuantumCircuit(3)
    if nombre == "AND":                # C empieza en 0; Toffoli lo invierte si A = B = 1
        qc.ccx(0, 1, 2)
    elif nombre == "OR":               # C empieza en 1; sale 0 solo si A = B = 0
        qc.x(2)
        qc.x([0, 1])
        qc.ccx(0, 1, 2)
        qc.x([0, 1])
    elif nombre == "NAND":             # AND y luego NOT a la salida
        qc.ccx(0, 1, 2)
        qc.x(2)
    elif nombre == "NOR":              # OR y luego NOT a la salida
        qc.compose(circuito_logico("OR"), inplace=True)
        qc.x(2)
    elif nombre == "XOR":              # C = A xor B: dos CNOT hacia C
        qc.cx(0, 2)
        qc.cx(1, 2)
    elif nombre == "XNOR":             # XOR y luego NOT a la salida
        qc.compose(circuito_logico("XOR"), inplace=True)
        qc.x(2)
    return qc


clasicas = {
    "AND": lambda a, b: a & b, "OR": lambda a, b: a | b,
    "NAND": lambda a, b: 1 - (a & b), "NOR": lambda a, b: 1 - (a | b),
    "XOR": lambda a, b: a ^ b, "XNOR": lambda a, b: 1 - (a ^ b),
}

# %% Celda 6: comprobar las tablas de verdad
# Para cada puerta, se pone A y B, se corre el circuito y se lee C. Para OR (y NOR)
# el qubit C debe empezar en 1 (lo hace el circuito mismo con la X inicial).
for nombre, f in clasicas.items():
    ok = True
    for a, b in itertools.product((0, 1), repeat=2):
        # Cadena q2 q1 q0 = "0" + str(b) + str(a), con C = q2 en 0.
        inicial = Statevector.from_label(f"0{b}{a}")
        probs = inicial.evolve(circuito_logico(nombre)).probabilities_dict()
        final = max(probs, key=probs.get)
        c = int(final[0])
        ok &= (c == f(a, b))
    print(f"{nombre:5s} coincide con la tabla de verdad clasica: {ok}")
```

1.5. Explicación celda por celda

Celda 2 Toma una unitaria al azar, calcula sus ángulos β, γ, δ con `OneQubitEulerDecomposer` y comprueba que se reconstruye con $R_z R_y R_z$.

Celda 3 Construye A, B, C con esos ángulos y verifica $ABC = I$ y $e^{i\alpha} AXBXC = U$.

Celda 4 Imprime la tabla de verdad de la Toffoli.

Celda 5 Define el circuito de cada puerta lógica.

Celda 6 Compara cada circuito con la tabla de verdad clásica, para las cuatro combinaciones de A y B.

1.6. Salida obtenida

```
U se reconstruye con Rz Ry Rz: True
angulos: beta = -4.179, gamma = 0.822, delta = 0.990
A B C = I      : True
e^{ia} A X B X C = U: True
Tabla de verdad (entrada q2q1q0 -> salida q2q1q0):
  000 -> 000
  001 -> 001
  010 -> 010
  011 -> 111
  100 -> 100
  101 -> 101
  110 -> 110
  111 -> 011
AND   coincide con la tabla de verdad clasica: True
OR    coincide con la tabla de verdad clasica: True
NAND  coincide con la tabla de verdad clasica: True
NOR   coincide con la tabla de verdad clasica: True
XOR   coincide con la tabla de verdad clasica: True
XNOR  coincide con la tabla de verdad clasica: True
```

Es determinista. Los ángulos dependen de la unitaria aleatoria elegida, que usa semilla fija.

1.7. Interpretación

Las dos descomposiciones se verifican (`True`). En la Toffoli, la cadena se lee como $q_2 q_1 q_0$: solo cambian $011 \to 111$ y $111 \to 011$, es decir, el objetivo q_2 se invierte cuando $q_0 = q_1 = 1$. Las seis puertas lógicas coinciden con su tabla de verdad clásica.

1.8. Ejercicios

1. Cambia la unitaria aleatoria (otra `seed`) y verifica de nuevo las dos descomposiciones.
2. Construye una puerta lógica con tres entradas, por ejemplo $C = A \wedge B \wedge D$ (necesita una puerta con tres controles).
3. ¿Por qué el circuito OR necesita que el objetivo empiece en $|1\rangle$?

1.9. Errores comunes

- Olvidar el orden de bits de Qiskit: la cadena `'011'` es $q_2 = 0$, $q_1 = 1$, $q_0 = 1$.
- Pensar que la puerta lógica clásica se aplica con una sola entrada: en la versión reversible las entradas se conservan (en A y B) y la salida se escribe en un qubit aparte.